

EXP NO : 1 THREE-LAYER NEURAL NETWORK FROM SCRATCH

CODE:

```
import matplotlib.pyplot as plt

import numpy as np

from sklearn.datasets import load_iris

from sklearn.model_selection import train_test_split

from sklearn.preprocessing import StandardScaler, OneHotEncoder

from tensorflow.keras.models import Sequential

from tensorflow.keras.layers import Dense, Input

iris = load_iris()

X = iris.data

y = iris.target.reshape(-1, 1)

classes = iris.target_names

encoder = OneHotEncoder(sparse=False)

y_encoded = encoder.fit_transform(y)

scaler = StandardScaler()

X_scaled = scaler.fit_transform(X)

X_train, X_test, y_train, y_test = train_test_split(

    X_scaled,

    y_encoded,

    test_size=0.2,

    random_state=42

)

print("Dataset Shape:", X.shape)

print("Training Data:", X_train.shape)
```

```

print("Testing Data:", X_test.shape)

def create_mlp(input_dim, output_dim, hidden_layers, activation='relu'):

    model = Sequential()

    model.add(Input(shape=(input_dim,)))

    for units in hidden_layers:

        model.add(Dense(units, activation=activation))

    model.add(Dense(output_dim, activation='softmax'))

    model.compile(

        optimizer='adam',

        loss='categorical_crossentropy',

        metrics=['accuracy']

    )

    return model

hidden_layer_configs = [

    [8],

    [16, 8],

    [32, 16, 8]

]

activations = [

    'relu',

    'tanh',

    'sigmoid'

]

results = []

last_model = None

last_history = None

for hidden_layers in hidden_layer_configs:

```

for activation in activations:

```
print("\nHidden Layers:", hidden_layers)

print("Activation:", activation)

model = create_mlp(

    input_dim=4,

    output_dim=3,

    hidden_layers=hidden_layers,

    activation=activation

)

history = model.fit(

    X_train,

    y_train,

    epochs=50,

    batch_size=5,

    verbose=0,

    validation_split=0.1

)

test_loss, test_acc = model.evaluate(

    X_test,

    y_test,

    verbose=0

)

print("Test Accuracy:", round(test_acc, 4))

print("Test Loss:", round(test_loss, 4))

results.append([

    hidden_layers,

    activation,
```

```
        test_acc,

        test_loss

    ])

    last_model = model

    last_history = history

print("\nMODEL COMPARISON")

for result in results:

    print(

        "Hidden Layers:", result[0],

        "| Activation:", result[1],

        "| Accuracy:", round(result[2], 4),

        "| Loss:", round(result[3], 4)

    )

plt.figure(figsize=(12, 5))

plt.subplot(1, 2, 1)

plt.plot(

    last_history.history['accuracy'],

    label='Training Accuracy'

)

plt.plot(

    last_history.history['val_accuracy'],

    label='Validation Accuracy'

)

plt.xlabel('Epoch')

plt.ylabel('Accuracy')

plt.title('MLP Training Accuracy')

plt.legend()
```

```
plt.grid(True)

plt.subplot(1, 2, 2)

plt.plot(

    last_history.history['loss'],

    label='Training Loss'

)

plt.plot(

    last_history.history['val_loss'],

    label='Validation Loss'

)

plt.xlabel('Epoch')

plt.ylabel('Loss')

plt.title('MLP Training Loss')

plt.legend()

plt.grid(True)

plt.tight_layout()

plt.show()

print("\nIRIS PREDICTION")

sepal_length = float(

    input("Enter Sepal Length (cm): ")

)

sepal_width = float(

    input("Enter Sepal Width (cm): ")

)

petal_length = float(

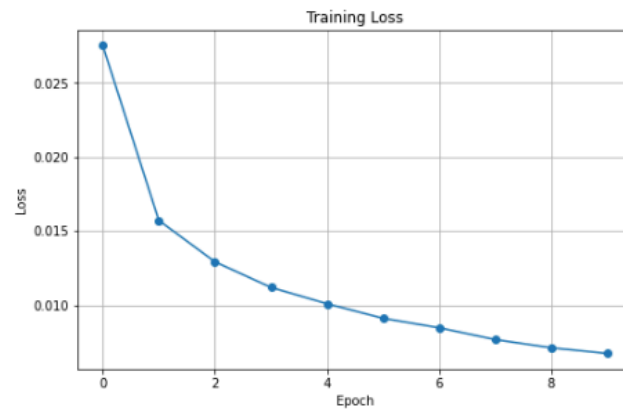
    input("Enter Petal Length (cm): ")

)
```

```
petal_width = float(
    input("Enter Petal Width (cm): ")
)
user_input = np.array([
    [
        sepal_length,
        sepal_width,
        petal_length,
        petal_width
    ]
])
user_input_scaled = scaler.transform(user_input)
prediction = last_model.predict(
    user_input_scaled,
    verbose=0
)
predicted_class_index = np.argmax(prediction)
predicted_class_name = classes[
    predicted_class_index
]
print(
    "\nPredicted Iris Species:",
    predicted_class_name
)
```

OUTPUT:

```
Training data shape: (60000, 784)
Training labels shape: (60000, 10)
Epoch 1/10 - Loss: 0.0275
Epoch 2/10 - Loss: 0.0157
Epoch 3/10 - Loss: 0.0129
Epoch 4/10 - Loss: 0.0112
Epoch 5/10 - Loss: 0.0101
Epoch 6/10 - Loss: 0.0091
Epoch 7/10 - Loss: 0.0085
Epoch 8/10 - Loss: 0.0077
Epoch 9/10 - Loss: 0.0071
Epoch 10/10 - Loss: 0.0067
```



Prediction: 5

